

PROJET UNIVERSITAIRE — SYSTÈMES DISTRIBUÉS

D-Ticket

*Système distribué de gestion de file d'attente
avec réplication et tolérance aux pannes*

Rapport de projet

Année académique 2025 – 2026

1. Introduction

Les systèmes de file d'attente (banques, administrations, hôpitaux) reposent sur une ressource critique : le **compteur de tickets**. Dans une architecture centralisée, ce compteur vit sur un serveur unique ; si ce serveur tombe, le service s'arrête et l'état peut être perdu.

D-Ticket répond à ce problème par une architecture **distribuée** : trois nœuds serveurs autonomes partagent et répliquent le compteur, si bien que la panne de l'un d'eux n'interrompt ni la distribution des tickets ni la conservation de l'historique.

2. Problématique

Comment garantir simultanément :

1. **l'unicité** des numéros de tickets (deux clients ne doivent jamais recevoir le même numéro) ;
2. **la disponibilité** du service quand un serveur tombe ;
3. **la durabilité** de l'état (aucun ticket perdu après un crash) ;

...sans recourir à une base de données centrale ni à un framework distribué lourd ?

3. Objectifs

- Concevoir un système avec **plusieurs agents autonomes** (3 nœuds).
- Distribuer et **répliquer** l'état métier (compteur + tickets) sur chaque nœud.
- Mettre en place un mécanisme de **synchronisation** (verrou local + versions).
- Assurer la **tolérance aux pannes** (failover automatique).
- Assurer la **récupération après crash** (resynchronisation au redémarrage).
- Offrir une **journalisation** et une **interface** de supervision.
- Valider le tout par des **tests système** automatisés.

4. Architecture distribuée

Le système comporte trois types de composants :

Composant	Port	Rôle
Registry	5000	Découverte des nœuds (annuaire) — aucun état métier
NODE_1, NODE_2, NODE_3	5001–5003	Agents autonomes : tickets, réplication, récupération
Interface web	8000	Supervision + routage des demandes vers un nœud actif

Chaque nœud est un **processus indépendant** avec sa propre mémoire et son propre fichier de persistance : il n'existe aucune mémoire partagée entre nœuds, toute coordination passe par des **messages HTTP** (bibliothèque `requests`). C'est la définition même d'un système distribué : des agents autonomes qui coopèrent par échange de messages pour maintenir un état global cohérent.

4.1 Rôle du registry

Le registry est le seul composant centralisé, et son rôle est volontairement minimal : c'est un **annuaire**.

- `POST /register` : un nœud s'annonce (et renouvelle son heartbeat toutes les 5 s) ;
- `GET /nodes` : liste des nœuds avec statut `active`/`inactive` (un nœud est déclaré inactif si aucun heartbeat depuis 15 s) ;
- `POST /unregister` : retrait explicite.

Le registry ne stocke **jamais** de tickets ni de compteur. S'il tombe, les nœuds déjà découverts continuent de fonctionner : seul l'ajout de nouveaux nœuds serait affecté. L'état métier n'est donc pas centralisé.

4.2 Rôle des nœuds

Chaque nœud maintient un état local persisté en JSON :

```
{
  "node_id": "NODE_1",
  "last_ticket": 4,
  "version": 4,
  "tickets": [ { "ticket_number": 1, "client": "Client A",
"created_by": "NODE_1", "timestamp": "..." } ],
  "event_log": [ { "timestamp": "...", "message": "..." } ]
}
```

- `last_ticket` : la ressource partagée (compteur) ;
- `version` : horloge logique simple, incrémentée à chaque mutation ;
- `tickets` : historique répliqué ;
- `event_log` : journal local (démarrages, pannes, réplifications, récupérations).

La création d'un ticket est **atomique** grâce à un `threading.Lock` : incrément du compteur, incrément de version, écriture disque, puis réplification.

5. Réplication

La réplication est de type **push, meilleure-effort** :

1. le nœud qui crée le ticket sérialise son état complet ;
2. il interroge le registry pour connaître les pairs actifs ;
3. il envoie son état à chaque pair via `POST /replicate` (timeout court : 2 s) ;
4. un pair injoignable est ignoré — il se resynchronisera à son retour.

Côté récepteur, la **règle de cohérence** est simple et sûre : l'état reçu n'est accepté que si sa `version` est **strictement supérieure** à la version locale. Un état obsolète ou dupliqué est donc toujours refusé, ce qui rend la réplication idempotente et protège contre les messages en retard.

6. Synchronisation

Deux mécanismes complémentaires :

- **Intra-nœud** : le `threading.Lock` sérialise les créations de tickets au sein d'un même processus. Deux requêtes simultanées obtiennent forcément deux numéros différents (vérifié par un test de concurrence à 10 threads).
- **Inter-nœuds** : le numéro de `version` ordonne les états. Tous les nœuds convergent vers la version maximale, celle du dernier ticket émis.

7. Tolérance aux pannes

La panne d'un nœud est simulée par `POST /shutdown-simulated` : le nœud répond alors `503` à toute requête métier et **cesse son heartbeat**, exactement comme un serveur éteint du point de vue du reste du système.

Le routage est géré par le backend web :

1. il sonde `GET /health` sur chaque nœud connu du registry ;

2. il répartit les demandes en **round-robin** entre les nœuds actifs **synchronisés** (version maximale), pour que tous les nœuds participent ;
3. si l'appel échoue malgré tout, il essaie le nœud suivant, les nœuds en retard n'étant utilisés qu'en dernier recours (risque de doublon).

Résultat : tant qu'**au moins un nœud** est vivant, les clients continuent de recevoir des tickets, avec des numéros toujours croissants et uniques.

8. Récupération après crash

Quand un nœud revient (`POST /restart-simulated` , ou simple relance du processus) :

1. il se ré-enregistre auprès du registry ;
2. il demande son état à chaque pair actif (`GET /state`) ;
3. il sélectionne l'état de **version la plus élevée** ;
4. si cette version dépasse la sienne, il l'adopte et la **persiste** en JSON ;
5. l'événement est consigné dans son journal.

Le nœud rattrape ainsi tous les tickets émis pendant son absence, sans intervention manuelle. La même procédure s'exécute au démarrage normal du processus, ce qui couvre aussi le cas d'un vrai crash (kill du processus).

9. Tests

Les tests (`pytest`) démarrent un **cluster réel** en sous-processus (registry, 3 nœuds avec dossier d'état temporaire, backend web) sur des ports libres :

Fichier	Ce qui est vérifié
<code>test_ticket_generation.py</code>	Premier ticket = n°1 ; numéros successifs distincts ; 10 requêtes concurrentes → 10 numéros uniques ; ticket via le backend web
<code>test_replication.py</code>	Un ticket créé sur NODE_1 apparaît sur NODE_2/NODE_3 ; convergence des versions ; rejet d'un état obsolète
<code>test_failover.py</code>	Nœud en panne → 503 ; le service continue via un autre nœud ; l'interface voit le nœud inactif
<code>test_recovery.py</code>	Un nœud redémarré récupère la dernière version ; <code>/recover</code> idempotent ; aucun doublon après récupération

10. Limites

- **Pas de consensus** (Raft/Paxos) : en cas de partition réseau où deux nœuds isolés recevraient chacun des demandes directes, un même numéro pourrait être émis deux fois. Dans notre déploiement, le backend web route vers un seul nœud à la fois, ce qui évite ce scénario en pratique.
- **Réplication de l'état complet** : simple, mais le volume transféré croît avec le nombre de tickets. Des deltas (envoyer seulement le nouveau ticket) seraient plus efficaces.
- **Registry unique** : point de défaillance pour la découverte (mais pas pour le service métier).
- **Cohérence à terme** : entre la création d'un ticket et la fin de la réplication, les nœuds peuvent être brièvement « en retard » (visible dans l'interface).

11. Améliorations futures

- Élection d'un **leader** (algorithme du plus petit identifiant, ou Bully) pour sérialiser globalement les créations de tickets.
- Réplication **incrémentale** (journal d'opérations plutôt qu'état complet).
- **Quorum d'écriture** (accusé de réception d'une majorité de nœuds avant de répondre au client) pour une durabilité plus forte.
- Registry répliqué ou découverte pair-à-pair (gossip).
- Authentification des échanges inter-nœuds.

12. Conclusion

D-Ticket démontre, avec une pile volontairement minimale (Python, Flask, JSON, HTTP), les mécanismes essentiels d'un système distribué : des agents autonomes coopérant par messages, un état répliqué et versionné, un service qui survit à la panne d'un nœud et un nœud qui se répare seul à son retour. La simplicité des choix (verrou local, réplication push, règle « version la plus haute gagne ») rend chaque mécanisme observable, testable et défendable, tout en laissant une voie d'évolution claire vers des techniques plus avancées (consensus, quorums).